

# ТЕХНИЧЕСКОЕ ЗАДАНИЕ И АРХИТЕКТУРНЫЙ РАЗБОР

ИНТЕРАКТИВНЫЙ КВИЗ-ЛЕНДИНГ «ЗОЛОТОЕ ХРАНИЛИЩЕ - ФИНДИРЕКТОР»

---

## 1. Общий анализ и концепция проекта

Предоставленный исходный код представляет собой современный интерактивный одностраничный сайт (Landing Page / SPA) с интегрированным маркетинговым квизом (опросником). Проект разработан с использованием концепции геймификации, где в качестве ключевого персонажа-эксперта выступает **Скрудж Макдак** (герой вселенной Disney, олицетворяющий богатство и жесткий финансовый учет).

**Целевая аудитория:** Владельцы малого и среднего бизнеса (товарный бизнес, услуги, производство, IT/Digital), у которых не настроен системный управленческий учет.

**Маркетинговая воронка сайта:**

1. **Захват внимания (Hero Section):** Крупное уникальное торговое предложение (УТП) + триггерное сравнение работы бухгалтера (смотрит в прошлое) и финдиректора (строит будущее).
2. **Вовлечение (Интерактивный квиз):** Опрос из 4 шагов, сегментирующий клиента по нише, масштабу выручки, ключевой боли и текущему методу учета.
3. **Персонализированная выдача (Калькулятор упущенной выгоды):** Математический расчет потенциальных потерь на основе реального ответа о выручке + выдача индивидуального текстового блока под конкретную проблему.
4. **Захват лида (Lead Capture Form):** Форма сбора контактных данных (Имя + Телефон/Telegram) для получения бесплатного финансового аудита.

## 2. Анализ технологического стека (С помощью чего это сделано)

Проект реализован в рамках **"No-Build" архитектуры (без этапа сборки)**. Это означает, что все внешние библиотеки, фреймворки и трансляторы кода подключаются напрямую через распределенные сети доставки контента (CDN). Весь проект полностью изолирован внутри одного файла `index.html`.

| Технология / Скрипт              | Тип подключения                              | Роль в архитектуре проекта                                                                                                                                                                                                |
|----------------------------------|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>React v18</b><br>(Production) | CDN ( <code>unpkg.com</code> )               | Ядро приложения. Отвечает за реактивное управление состоянием интерфейса (State Management), переключение шагов квиза и рендеринг динамических блоков без перезагрузки страницы.                                          |
| <b>ReactDOM v18</b>              | CDN ( <code>unpkg.com</code> )               | Связующее звено, которое монтирует виртуальное дерево компонентов React в реальный DOM-узел шаблона ( <code>&lt;div id="root"&gt;</code> ).                                                                               |
| <b>Babel Standalone</b>          | CDN ( <code>unpkg.com</code> )               | Транспилиратор кода. Позволяет браузеру читать синтаксис JSX и современные стандарты ES6+ "на лету" прямо во время загрузки страницы, компилируя тег <code>&lt;script type="text/babel"&gt;</code> .                      |
| <b>Tailwind CSS</b>              | CDN<br>( <code>cdn.tailwindcss.com</code> )  | Атомарный (Utility-first) CSS-фреймворк. Позволяет мгновенно стилизовать элементы с помощью классов прямо в разметке, обеспечивая адаптивность (классы типа <code>md:grid-cols-2</code> , <code>hidden sm:block</code> ). |
| <b>Font Awesome v6.4.0</b>       | CDN<br>( <code>cdnjs.cloudflare.com</code> ) | Библиотека векторных иконок (иконки банка, кошелек, графиков, калькулятора, стрелок).                                                                                                                                     |

#### Важное техническое замечание для Production-среды:

Использование `Babel Standalone` на стороне клиента отлично подходит для быстрого прототипирования, тестирования гипотез (MVP) и пет-проектов. Однако для реального продакшена с высокой посещаемостью это замедляет скорость первой отрисовки (LCP), так как браузер тратит ресурсы процессора на компиляцию JS-кода в реальном времени. В будущем рекомендуется перенести проект на сборщик типа `Vite`.

### 3. Пошаговое ТЗ: Как воссоздать этот проект с нуля

Ниже описана детальная последовательность шагов разработки, воспроизводящая логику и дизайн данного коммерческого предложения.

#### Шаг 1: Разработка структуры документа и подключение вендоров

Создается базовый HTML5-каркас. В теге `<head>` подключаются необходимые стили и скрипты (Tailwind, Font Awesome, React, ReactDOM, Babel). Внутри `<body>` создается единственный пустой контейнер `<div id="root"></div>`. Вся последующая верстка пишется внутри тега `<script type="text/babel">` .

## Шаг 2: Проектирование State (Состояния приложения)

В главном функциональном компоненте `App()` инициализируются базовые хуки управления состоянием:

- `quizStarted` (boolean) — запущен ли опрос.
- `currentStep` (number) — индекс текущего вопроса квиза (от 0 до 3).
- `showResult` (boolean) — триггер для отображения финального коммерческого предложения.
- `answers` (object) — структура хранения выбранных пользователем данных: `{ niche, revenue, revenueLabel, pain, current }`.

## Шаг 3: Верстка шапки и Главного экрана (Hero Section)

Используя утилитарные классы Tailwind CSS, создается темная стильная шапка (`bg-slate-900`) с фиксированным позиционированием (`sticky top-0 z-50`). Главный экран разделяется на две колонки через сетку (`grid md:grid-cols-2`):

- **Левая колонка:** Заголовки с акцентами желтого цвета (`text-yellow-400`), маркетинговый текст и кнопка СТА, которая переключает `quizStarted` в значение `true`.
- **Правая колонка:** Информационная карточка-сравнение функций бухгалтера и финдиректора, декоративный спич-бабл (облако мыслей) с приветственной цитатой Скруджа Макдака и аватар персонажа, загружаемый со стороннего фотохостинга.

## Шаг 4: Реализация интерактивного квиза (Пошаговая логика)

Секция квиза рендерится всегда, но контент внутри зависит от переменной `currentStep`. Реализован индикатор прогресса (ProgressBar), ширина которого динамически вычисляется формулой: `style={{ width: `${((currentStep) / 4) * 100}%` }}`.

С помощью условного рендеринга React (`{currentStep === X && (...)}`) пользователю поочередно выводятся 4 экрана:

### Вопросы и структура собираемых данных:

1. **Ниша бизнеса:** Кнопки выбора (Товарный бизнес, Сфера услуг, Производство, IT и Digital). Пишется в поле `niche`.
2. **Масштаб империи (Выручка):** Числовые маркеры для расчетов (до 1 млн -> 1 000 000, 1-5 млн -> 3 000 000, 5-15 млн -> 10 000 000, более 15 млн -> 20 000 000). Пишется в поле `revenue`.
3. **Главная боль:** Варианты («кассовые разрывы», «невидимая прибыль», «смешивание карманов», «страх роста»). Пишется в поле `pain`.
4. **Текущее состояние учета:** Выбор текущего ответственного (Я сам, Бухгалтер, Хаос в таблицах). Пишется в поле `current`.

При клике на любой вариант срабатывает функция `handleAnswer()`, которая обновляет стейт ответов и инкрементирует шаг (`currentStep + 1`). На последнем шаге переключается флаг `showResult = true`.

#### Шаг 5: Программирование алгоритма калькулятора и динамической выдачи

В коде прописана 핵심-логика интерактивного калькулятора потерь. Метод `calculateLoss()` использует константную ставку скрытых бизнес-потерь в размере 8% (`0.08`):

```
const lossPercentage = 0.08;
const monthlyLoss = answers.revenue * lossPercentage;
const yearlyLoss = monthlyLoss * 12;
return new Intl.NumberFormat('ru-RU').format(yearlyLoss);
```

Функция `getPersonalizedContent()` содержит конструкцию `switch(answers.pain)`. Она сопоставляет выбранную боль пользователя с кастомными строками данных: генерирует индивидуальный заголовок решения, подробное описание методологии внедрения (например, платежный календарь для кассовых разрывов или P&L-отчет для невидимой прибыли) и уникальную, стилизованную под Скруджа Макдака, реплику.

#### Шаг 6: Плавный скролл и UX-оптимизация

Чтобы пользователь не терялся при переключении экранов, задействован хук `useRef`. Созданы два рефа: `quizRef` и `resultRef`. Функция `scrollToRef()` осуществляет плавный автоматический скролл окна браузера к началу нужного блока при старте квиза и при выводе оффера.

#### Шаг 7: Захват контактных данных и лидогенерация

В самом низу динамического блока результатов выводится золотая СТА-карточка с формой сбора контактов. Метод обработки отправки формы имеет заглушку `e.preventDefault()` и выводит демонстрационное всплывающее окно `alert()`. В реальном production-приложении этот обработчик отправляет JSON-пакет данных с ответами квиза и контактами в CRM-систему или Telegram Webhook.

## 4. Рекомендации по масштабированию и улучшению кода

Данный код представляет собой отличный фундамент для высококонверсионного коммерческого предложения. Чтобы превратить его в коммерческий продукт корпоративного уровня, рекомендуется провести следующие улучшения:

#### 1. Интеграция с бэкендом (Отправка данных):

Заменить стандартный `alert()` на отправку AJAX-запроса через `fetch()`. Настроить интеграцию с CRM (amoCRM, Bitrix24) или напрямую отправлять уведомления в Telegram-канал отдела продаж через Telegram Bot API.

## **2. Улучшение UX калькулятора:**

Вместо жестко фиксированных кнопок выбора выручки на Шаге 2 (где берутся средние значения 1, 3, 10 или 20 млн), можно внедрить интерактивный ползунок-слайдер ( `<input type="range">` ). Это позволит пользователю указать точную сумму своей выручки, делая расчет годовых потерь абсолютно точным и вызывающим больше доверия.

## **3. Оптимизация медиа-ресурсов:**

Изображения Скруджа Макдака сейчас загружаются с внешнего бесплатного хостинга картинок Yарх. Для стабильной работы и защиты от удаления файлов изображения следует скачать локально в проект или загрузить в облачное хранилище (S3 / Cloudinary) с гарантированным uptime.